

AI-Assisted Client Website Workflow

Objective

Create three website concepts (P1, P2, P3) for a client using a single repository, shared assets, and AI coding agents.

The goal is to quickly present multiple design directions while maintaining a clean project structure and minimizing deployment complexity.

Step 1: Gather Client Information

Collect:

- Specification document
 - Logo files
 - Images
 - Brand assets
 - Business information
 - Contact details
 - Social media links
 - Preferred colours
 - Competitor examples
 - Website inspirations
-

Step 2: Create Project Structure

```
client-website/
|
├── README.md
|
├── context/
|   ├── specification-doc.pdf
|   ├── extracted-requirements.md
|   └── agent-instructions.md
|
├── assets/
|   ├── logo/
|   ├── images/
|   └── icons/
```

```
|   └─ branding/
|
|   └─ p1/
|
|   └─ p2/
|
|   └─ p3/
|
|   └─ index.html
```

Step 3: Create an Extracted Requirements Document

Do not rely solely on the original specification document.

Create:

```
context/extracted-requirements.md
```

Template:

```
Business Name:
```

```
Business Description:
```

```
Website Goal:
```

```
Target Audience:
```

```
Required Pages:
```

```
Required Sections:
```

```
Primary Call To Action:
```

```
Services Offered:
```

```
Brand Personality:
```

```
Preferred Colours:
```

Preferred Style:

Things To Avoid:

Contact Information:

Social Media Links:

Assets Available:

Special Requirements:

This becomes the primary source of truth for all agents.

Step 4: Create Design Directions

Before generating websites, define three distinct concepts.

Example:

P1

Modern and clean
Simple navigation
Mobile-first
Professional

P2

Warm and welcoming
Community focused
Friendly personality
Story-driven

P3

Premium and high-end
Luxury presentation
Strong visual impact
Elegant typography

Step 5: Launch AI Agents

Agent 1

Read everything inside /context.

Use assets from /assets.

Create Prototype P1.

Only modify files inside /p1.

Build a static HTML/CSS website.

Agent 2

Read everything inside /context.

Use assets from /assets.

Create Prototype P2.

Only modify files inside /p2.

Build a static HTML/CSS website.

Agent 3

Read everything inside /context.

Use assets from /assets.

Create Prototype P3.

Only modify files inside /p3.

Build a static HTML/CSS website.

Step 6: Create Review Portal

Root homepage:

```
<h1>Website Concepts</h1>

<a href="/p1/">Prototype 1</a>
<a href="/p2/">Prototype 2</a>
<a href="/p3/">Prototype 3</a>
```

This acts as a concept selection page.

Step 7: Local Review

Review each prototype for:

- Broken links
- Missing assets
- Mobile responsiveness
- Brand consistency
- Copy quality
- Contact information accuracy

Make corrections before showing the client.

Step 8: Deploy

Deploy the repository once.

Example:

```
clientwebsite.vercel.app
```

Client URLs:

```
clientwebsite.vercel.app/p1
clientwebsite.vercel.app/p2
clientwebsite.vercel.app/p3
```

No separate repositories required.

No separate hosting projects required.

Step 9: Client Feedback

Ask the client:

Favourite Concept

P1 / P2 / P3

Favourite Elements

Colours
Layout
Typography
Images
Navigation

Requested Changes

Add
Remove
Modify

Step 10: Final Development

Create:

final/

or replace:

p1/

with the selected concept.

Implement feedback.

Refine:

- Accessibility
- SEO
- Performance
- Mobile experience
- Copywriting
- Contact forms

Deliverables

Final repository contains:

```
context/  
assets/  
final/  
README.md
```

Client receives:

- Live website
- Source code repository
- Domain connection
- Documentation
- Asset package

Advantages

- One repository
- One deployment
- Shared assets
- Shared requirements
- Easy client review process
- Multiple design options
- Fast AI-assisted generation
- Minimal hosting costs
- Scales well for future client projects